```
<a href="#"
onclick="this.style.backgroundColor='#009900';this.style.color='#FFFFFF';">Paint it
green</a>
```

Awkward, right? We could pull the function definition out of the click handler, and turn it into CoffeeScript:

```
paintIt = (element, backgroundColor, textColor) ->
  element.style.backgroundColor = backgroundColor
  if textColor?
    element.style.color = textColor
```

And then on our page:

```
<a href="#" onclick="paintIt(this, '#990000')">Paint it red</a>
```

That's a little bit better, but what about multiple links that have the same effect?

```
<a href="#" onclick="paintIt(this, '#990000')">Paint it red</a>
<a href="#" onclick="paintIt(this, '#009900', '#FFFFFF')">Paint it
green</a>
<a href="#" onclick="paintIt(this, '#000099', '#FFFFFF')">Paint it
blue</a>
```

Not very DRY, eh? We can fix this by using events instead. We'll add a `data-*` attribute to our link, and then bind a handler to the click event of every link that has that attribute:

```
paintIt = (element, backgroundColor, textColor) ->
  element.style.backgroundColor = backgroundColor
  if textColor?
    element.style.color = textColor

$ ->
  $("a[data-background-color]").click (e) ->
    e.preventDefault()

    backgroundColor = $(this).data("background-color")
    textColor = $(this).data("text-color")
    paintIt(this, backgroundColor, textColor)
<a href="#" data-background-color="#990000">Paint it red</a>
<a href="#" data-background-color="#009900" data-text-
color="#FFFFFF">Paint it green</a>
<a href="#" data-background-color="#000099" data-text-
color="#FFFFFF">Paint it blue</a>
```

We call this 'unobtrusive' JavaScript because we're no longer mixing our JavaScript into our HTML. We've properly separated our concerns, making future change easy. We can easily add behavior to any link by adding the data attribute. We can run all of our JavaScript through a minimizer and concatenator. We can serve our entire JavaScript bundle on every page, which means that it'll get downloaded on the first page load and then be cached on every page after that. Lots of little benefits really add up.

The Rails team strongly encourages you to write your CoffeeScript (and JavaScript) in this style, and you can expect that many libraries will also follow this pattern.

3 Built-in Helpers

Rails provides a bunch of view helper methods written in Ruby to assist you in generating HTML. Sometimes, you want to add a little Ajax to those elements, and Rails has got your back in those cases.

Because of Unobtrusive JavaScript, the Rails "Ajax helpers" are actually in two parts: the JavaScript half and the Ruby half.